

Assignment 5 – I2C kommunikasjon mellom to ATmega32

Oskar Hellebust-Nygaard

26. mars 2026

Innhold

1	Innledning	2
1.1	Endring fra opprinnelig løsning	2
2	Kobling	2
3	Teori	3
3.1	I2C-prinsipp	3
3.2	Slaveadresse	4
3.3	TWI-hastighet	4
4	Slave – polling-basert løsning	4
4.1	Prinsipp	4
4.2	Knappehåndtering	4
4.3	TWI-statusbehandling i slave	5
5	Master – interrupt-basert løsning	5
5.1	Interrupt-oppsett	5
5.2	TWI-statusbehandling i master	5
6	i2c.h	6
7	Programkode	6
7.1	Slave-kode (lærerens kode)	6
7.2	Master-kode	7
7.3	i2c.h	9
8	Resultat	10
9	Feilkilder	11

1 Innledning

I denne oppgaven ble to ATmega32 koblet sammen med I2C (TWI), der en fungerer som slave og en som master.

Begge mikrokontrollerne har:

- 2 LED-er
- 2 knapper

Slaven skal toggle sine egne LED-er med egne knapper. Masteren skal ved knappetrykk lese statusen til slavens LED-er og kopiere det til sine egne LED-er.

Kravene var:

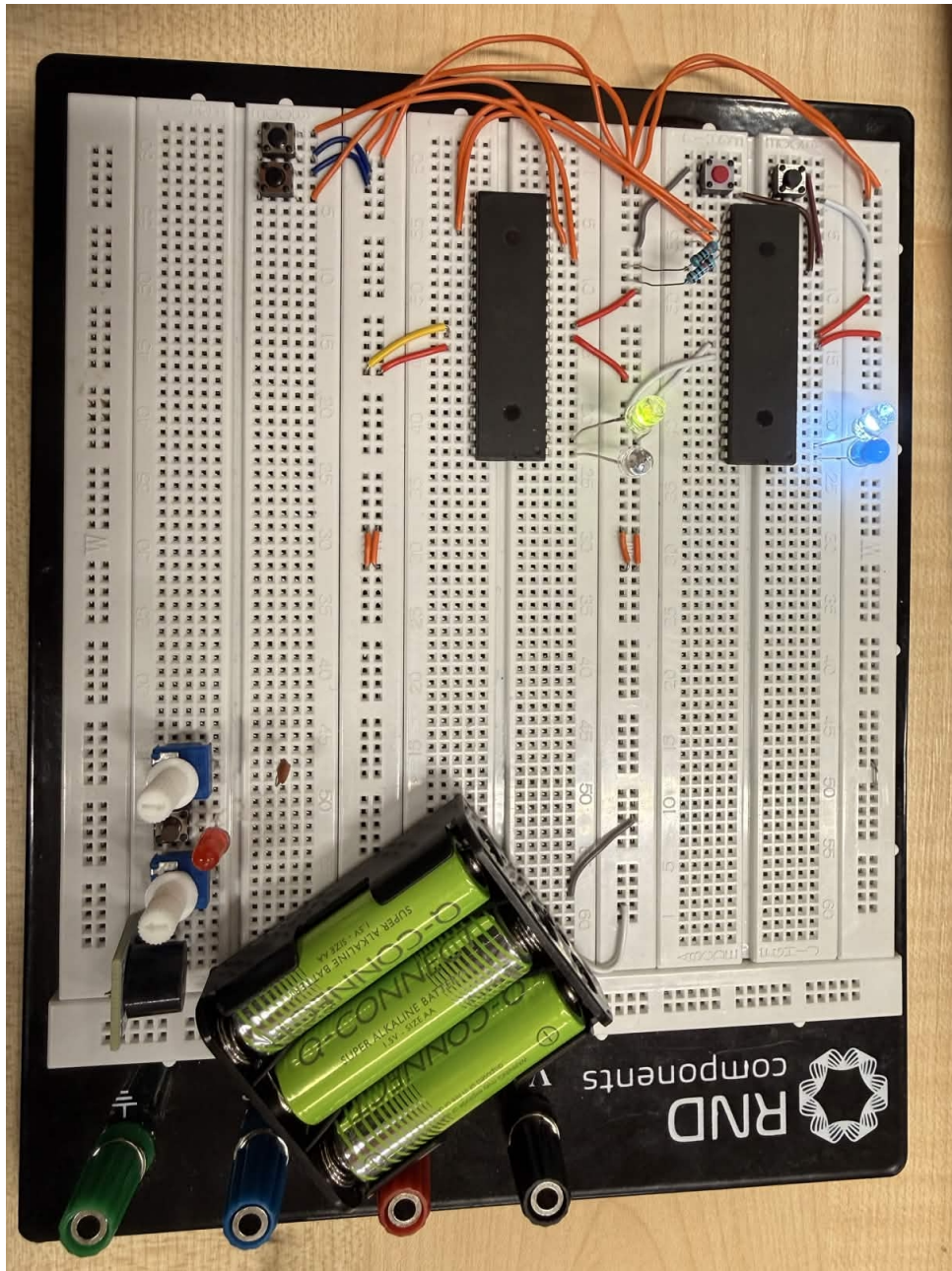
- Interrupt skal brukes
- Event-loop skal være tom
- Egen `i2c.h` headerfil skal brukes
- Switch-case skal brukes i TWI-statusbehandling
- Alle master-statuskoder unntatt `0x50` skal håndteres
- Slavekoden gitt skal brukes

1.1 Endring fra opprinnelig løsning

Etter å ha levert en første versjon av oppgaven fikk jeg tilbakemelding fra lærer om at jeg skulle bruke den oppgitte slavekoden i stedet for min egen interrupt-baserte slave. Den opprinnelige slavekoden brukte TWI-interrupt (`ISR(TWI_vect)`) og håndterte statuskodene `0xA8`, `0xB8`, `0xC0` og `0xC8` med `switch-case`. Lærers slavekode er polling-basert: den sjekker `TWSR` direkte i main-loopen og venter på status `0xA8` (SLA+R mottatt). Slaveknappen er koblet til PB1 og bruker debounce i stedet for ekstern interrupt. Resten av systemet master og `i2c.h` er uendret.

2 Kobling

- PB0 til LED og GND (master: PB0 og PB1)
- Slave: PB1 til trykknapp mot GND, intern pull-up
- Master: PD2 og PD3 til trykknapper mot GND, intern pull-up
- PC1 = SDA mellom begge kontrollere
- PC0 = SCL mellom begge kontrollere
- Felles GND mellom begge mikrokontrollere
- Pull-up motstand på SDA og SCL (I2C er open-drain)



Figur 1: Oppkobling av master og slave med I2C

3 Teori

3.1 I2C-prinsipp

I2C bruker to ledninger: SDA (data) og SCL (klokke). Master kontrollerer alltid klokken. Kommunikasjonssekvensen er:

1. Master sender START
2. Master sender slaveadresse + R/W-bit
3. Data sendes eller mottas

4. Master sender STOP

3.2 Slaveadresse

Slaven fikk adresse 18. I læreren kode settes dette direkte som 7-bits adresse i de 7 MSB av TWAR:

$$TWAR = 0b00100100$$

Dette tilsvarer adresse 18 shiftet ett hakk til venstre. Lærerens kode bruker en direkte binær verdi i stedet for $(18 \ll 1)$.

3.3 TWI-hastighet

Master bruker:

$$TWBR = 2, \quad TWSR = 0x00 \quad (\text{prescaler} = 1)$$

Bitrate-formel:

$$f_{SCL} = \frac{F_{CPU}}{16 + 2 \cdot TWBR \cdot 4^{TWPS}} = \frac{1\,000\,000}{16 + 2 \cdot 2 \cdot 1} = 50\,000 \text{ Hz} = 50 \text{ kHz}$$

4 Slave – polling-basert løsning

4.1 Prinsipp

Lærerens slavekode er polling-basert. Det betyr at slaven ikke bruker TWI-interrupt; i stedet sjekkes TWSR kontinuerlig i main:

```
if (TWSR == 0xA8)    // SLA+R mottatt
```

Når master ber om data, lastes LEDStatus inn i TWDR og sendes. Etter overføring switcher slaven tilbake til not-addressed slave mode ved å sette TWEA og TWEN.

4.2 Knappehåndtering

Slaven har en knapp på PB1 med intern pull-up. Knappen debounces med en enkel forsinkelsesbasert funksjon:

```
uint8_t debounce(char pinName, uint8_t pinNumber)
{
    if (bit_is_clear(pinName, pinNumber))
    {
        _delay_ms(5);
        if (bit_is_clear(pinName, pinNumber))
            return (1);
    }
    return (0);
}
```

LED-status lagres i variabelen LEDStatus:

- 0xF0 = LED av
- 0xF1 = LED på

4.3 TWI-statusbehandling i slave

Slaven håndterer to TWI-statuser polling-basert:

- 0xA8 – SLA+R mottatt: last LEDStatus i TWDR og send
- 0xC0 – data sendt med NACK: switch tilbake til not-addressed slave mode

5 Master – interrupt-basert løsning

5.1 Interrupt-oppsett

Master bruker ekstern interrupt på INT0 og INT1 (PD2 og PD3) med falling edge:

```
MCUCR |= (1 << ISC01) | (1 << ISC11);  
MCUCR &= ~( (1 << ISC00) | (1 << ISC10) );
```

Ved knappetrykk settes `buttonRequest` og START sendes umiddelbart.

5.2 TWI-statusbehandling i master

All TWI-logikk skjer i `ISR(TWI_vect)` med switch-case. Statuskodene som håndteres:

- 0x08 / 0x10 – START sendt: send SLA+R
- 0x40 – SLA+R sendt, ACK: gjør klar til å motta 1 byte med NOT_ACK
- 0x58 – data mottatt, NACK: les TWDR, oppdater LED, send STOP
- 0x18, 0x20, 0x28, 0x30, 0x48 – uventede tilstander: send STOP
- 0x38 – arbitration lost: send ny START

Når data er mottatt sjekkes `slaveData` mot 0xF1 for å avgjøre om LED skal lyse:

```
case 0x58:  
    slaveData = TWDR;  
    switch (buttonRequest)  
    {  
        case 1:  
            if (slaveData == 0xF1)  
                PORTB |= (1 << PB0);  
            else  
                PORTB &= ~(1 << PB0);  
            break;  
        case 2:  
            if (slaveData == 0xF1)  
                PORTB |= (1 << PB1);  
            else  
                PORTB &= ~(1 << PB1);  
            break;  
    }  
    sendSTOP();  
    break;
```

6 i2c.h

Headerfilen samler alle nødvendige TWI-funksjoner som `static inline`:

- `sendStart()` – sender START-betingelse
- `sendSLA(address, read_write)` – sender slaveadresse med R/W-bit
- `sendDATA(data, last_not_last)` – sender data, med eller uten TWEA
- `sendSTOP()` – sender STOP-betingelse
- `receiveDATA(ack_not_ack)` – gjør klar til mottak med ACK eller NACK
- `switchNASM(recognize)` – bytter til not-addressed slave mode

7 Programkode

7.1 Slave-kode (lærerens kode)

```
/*
 * I2C Slave.c - Lærers kode
 */

#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>

uint8_t debounce(char pinName, uint8_t pinNumber)
{
    if (bit_is_clear(pinName, pinNumber))
    {
        _delay_ms(5);
        if (bit_is_clear(pinName, pinNumber))
            return (1);
    }
    return (0);
}

int main(void)
{
    DDRB |= 1 << PINB0;           // PB0 som output
    PORTB |= 1 << PINB1;           // Pull-up på PB1

    TWAR = 0b00100100;             // Slaveadresse 18 (7 MSB)
    TWCR = (1 << TWEA) | (1 << TWEN); // TWI på, ACK på

    uint8_t LEDStatus = 0xF0;      // LED av
    uint8_t buttonPressedInstructions = 0;

    while (1)
    {
```

```

    if (debounce(PINB, 1))
    {
        if (buttonPressedInstructions == 0)
        {
            if (LEDStatus == 0xF0)
            {
                PORTB |= 1 << PB0;
                LEDStatus = 0xF1;
            }
            else
            {
                PORTB &= ~(1 << PB0);
                LEDStatus = 0xF0;
            }
            buttonPressedInstructions = 1;
        }
    }
    else
    {
        buttonPressedInstructions = 0;
    }

    if (TWSR == 0xA8) // SLA+R mottatt
    {
        TWDR = LEDStatus;
        TWCR = (1 << TWINT) | (1 << TWEN); // Send LEDStatus
        while ((TWCR & (1 << TWINT)) == 0); // Vent til ferdig
        if (TWSR == 0xC0) // Data sendt, NACK
        {
            TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWEA); //
            Tilbake til NASM
        }
    }
}
}

```

7.2 Master-kode

```

#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include "i2c.h"

volatile uint8_t buttonRequest = 0;
volatile uint8_t slaveData = 0;

ISR(INT0_vect)
{
    buttonRequest = 1;
    sendStart();
}

```

```

}

ISR(INT1_vect)
{
    buttonRequest = 2;
    sendStart();
}

ISR(TWI_vect)
{
    switch (TWSR & 0xF8)
    {
        case 0x08: // START sendt
        case 0x10: // Repeated START sendt
            sendSLA(18, READ);
            break;

        case 0x40: // SLA+R sendt, ACK
            receiveDATA(NOT_ACK);
            break;

        case 0x58: // Data mottatt, NACK
            slaveData = TWDR;
            switch (buttonRequest)
            {
                case 1:
                    if (slaveData == 0xF1)
                        PORTB |= (1 << PB0);
                    else
                        PORTB &= ~(1 << PB0);
                    break;
                case 2:
                    if (slaveData == 0xF1)
                        PORTB |= (1 << PB1);
                    else
                        PORTB &= ~(1 << PB1);
                    break;
                default:
                    break;
            }
            buttonRequest = 0;
            sendSTOP();
            break;

        case 0x18: // SLA+W sendt, ACK
        case 0x20: // SLA+W sendt, NACK
        case 0x28: // Data sendt, ACK
        case 0x30: // Data sendt, NACK
            sendSTOP();
            break;
    }
}

```



```

        case 0x38: // Arbitration lost
            sendStart();
            break;

        case 0x48: // SLA+R sendt, NACK
            sendSTOP();
            break;

        default:
            sendSTOP();
            break;
    }
}

int main(void)
{
    DDRB |= (1 << PB0) | (1 << PB1);
    DDRD &= ~((1 << PD2) | (1 << PD3));
    PORTD |= (1 << PD2) | (1 << PD3);

    TWBR = 2;
    TWSR = 0x00;

    GICR |= (1 << INT0) | (1 << INT1);
    MCUCR |= (1 << ISC01) | (1 << ISC11);
    MCUCR &= ~((1 << ISC00) | (1 << ISC10));

    TWCR = (1 << TWEN) | (1 << TWIE);

    sei();
    while (1) {}
}

```

7.3 i2c.h

```

#ifndef I2C_H_
#define I2C_H_

#include <avr/io.h>

#define WRITE      0
#define READ       1
#define ACK        1
#define NOT_ACK    0
#define LAST       1
#define NOT_LAST   0

static inline void sendStart(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN) | (1 << TWIE);
}

```

```

}

static inline void sendSLA(uint8_t address, uint8_t read_write)
{
    TWDR = (address << 1) | read_write;
    TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
}

static inline void sendData(uint8_t data, uint8_t last_not_last)
{
    TWDR = data;
    if (last_not_last == LAST)
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
    else
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) | (1 <<
            TWEA);
}

static inline void sendSTOP(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN) | (1 << TWIE);
}

static inline void receiveData(uint8_t ack_not_ack)
{
    if (ack_not_ack == ACK)
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) | (1 <<
            TWEA);
    else
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
}

static inline void switchNASM(uint8_t recognize)
{
    if (recognize)
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) | (1 <<
            TWEA);
    else
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
}

#endif /* I2C_H_ */

```

8 Resultat

Systemet fungerer slik:

- Slave-knapp (PB1) toggler slave-LED (PB0) via debounce i main
- Master INT0 leser slavestatus og kopierer til master PB0

- Master INT1 leser slavestatus og kopierer til master PB1
- Event-loop er tom på master

Siden lærerens slave bare har en LED og sender 0xF1 (på) eller 0xF0 (av), sjekker master mot disse verdiene direkte i stedet for bitmasking.

9 Feilkilder

- PD2 var ved en anledning koblet til VCC i stedet for GND, noe som hindret falling edge interrupt fra å trigge
- Flere knapper måtte byttes ut da de ikke ga pålitelig kontakt
- Startet med MISO og MOSI, men innså at disse kun brukes til SPI byttet til PC0/PC1 for I2C